NOVA RAG Pipeline

End-to-End Metadata Walkthrough

How Metadata Flows from Corpus to Answer

Two Real Documents | Five Pipeline Stages | Three Metadata Rules

March 2026

1. What We Have: The OSFI Corpus

The NOVA corpus contains 278 OSFI regulatory guidance documents scraped from the live Guidance Library. Each document exists in four formats, plus a master metadata index.

File Inventory

Directory	Files	Purpose
json/	278	Canonical source: full metadata (121 fields) + structured content (sections/paragraphs/lists/tables)
html/	278	Raw scraped HTML with <meta> tags for the parse_html() ingestion path
md/	278	Markdown renderings for human readability and review
pdf/	277	PDF copies for archival reference
osfi_guidance_metadata.json	1	Master metadata index: all 278 documents with 121 fields each

Document Breakdown

Dimension	Values
Status	final_current (244), superseded (20), final_future_effective (9), draft_or_consultation (5)
NOVA Tier	Tier 1 (151), Tier 2 (121), Tier 3 (6)
Document Class	transmittal_letter (108), package_guideline (44), chapter_guideline (36), advisory (32), standalone_guideline (27), other (31)

2. The Three Rules for Metadata Treatment

Every metadata field has a specific job in the pipeline. The Three Rules prevent the common failure mode where fields are used in the wrong place, producing retrieval that looks correct but generates inaccurate answers.

The Core Insight

Embedding, filtering, and prompt injection are three completely separate mechanisms serving three completely different purposes. Confusing them produces subtle accuracy failures that are hard to diagnose because the retrieval metrics look fine but the model's answers are undated or incorrectly attributed.

Rule	Purpose	Where It Happens	What Goes Here
Rule 1: Embed	Change where the chunk sits in vector space	Prepended to chunk text before embedding	regulator, short_title, guideline_number, document_class, heading_path, section_path
Rule 2: Index	Gate and boost retrieval without affecting semantics	Stored as filter/boost fields in ES and PGVector	status, nova_tier, effective_date_start/end, jurisdiction, authority_class, sector, current_version_flag, supersedes/superseded_by
Rule 3: Prompt-inject	Give the LLM context to reason correctly	Rendered into the prompt header at answer time	title, version_label, status, effective_date_start/end, authority_class, nova_tier, jurisdiction, citation_anchor, regulator
Operational	Lineage, auditing, reprocessing	Stored in data lake only; never shown to the model	raw_path, canonical_json_path, raw_sha256, parser_version, quality_score, quality_flags, download_timestamp

3. Our Two Example Documents

To show how metadata behaves differently depending on the document, we will trace two contrasting examples through every pipeline stage.

Document A: LAR (2026) Chapter 2 – Liquidity Coverage Ratio

Field	Value
doc_id	osfi.lar.2026.chapter2.chapter
document_class	chapter_guideline
nova_tier	1 (primary normative)
authority_class	primary_normative
authority_level	100
status	final_future_effective
effective_date_start	2026-05-01
guideline_number	LAR
sector	Banks, Trust and Loan Companies
sections	18 sections, 550 content items

This is a dense, chapter-level prudential guideline. It defines the LCR formula, HQLA definitions, and cash flow calculation rules. Tier 1 means it gets the highest retrieval priority.

Document B: MCT – Letter (2023)

Field	Value
doc_id	osfi.mct.2023.transmittal
document_class	transmittal_letter
nova_tier	2 (official support)
authority_class	official_support
authority_level	60
status	superseded
effective_date_start	2023-10-20
superseded_by_doc_id	osfi.mct.2026.transmittal
guideline_number	MCT
sector	Property and Casualty Companies
sections	2 sections, 185 words total

This is a short transmittal letter announcing the MCT 2024 guideline. It has been superseded and is Tier 2, so it serves as context but should not override current guidance.

4. Stage-by-Stage Pipeline Walkthrough

The pipeline has five stages. At each stage, specific metadata fields are consumed for specific purposes. Here is exactly what happens to our two documents.

Stage 1: Parsing — From File to CanonicalDocument

The parser (parse_osfi_canonical_json or parse_html) reads the raw file and creates a CanonicalDocument object with a list of CanonicalUnit objects. This is where structured content gets paired with document-level metadata.

What the parser reads from the JSON file:

Document A (LAR Chapter 2) — The parser walks the sections array. Each paragraph and list becomes a CanonicalUnit with its own heading_path and section_path computed from the section structure.

```
parse_osfi_canonical_json("bronze/osfi/json/osfi.lar.2026.chapter2.chapter.json")

Result: CanonicalDocument with:
  doc_id    = "osfi.lar.2026.chapter2.chapter"
  title     = "Liquidity Adequacy Requirements (LAR) (2026) Chapter 2..."
  status    = "final_future_effective"
  nova_tier = 1
  units     = [550 CanonicalUnit objects]

Sample unit:
  unit_id      = "osfi.lar.2026.chapter2.chapter::sec3::p2"
  heading_path = ["LAR (2026) Ch 2", "2.1. Objective of the LCR"]
  section_path = "LAR (2026) Ch 2 > 2.1. Objective of the LCR"
  text         = "This standard aims to ensure that an institution has an"
                 "adequate stock of unencumbered HQLA..."
```

Document B (MCT Letter) — The parser does the same, but yields far fewer units because the letter is only 185 words.

```
parse_osfi_canonical_json("bronze/osfi/json/osfi.mct.2023.transmittal.json")

Result: CanonicalDocument with:
  doc_id    = "osfi.mct.2023.transmittal"
  status    = "superseded"
  nova_tier = 2
  superseded_by_doc_id = "osfi.mct.2026.transmittal"
  units     = [6 CanonicalUnit objects]
```

Metadata used at this stage

All 121 JSON fields are read, but the critical ones are: doc_id, title, status, effective_date_start, nova_tier, authority_class, guideline_number, sector, supersedes_doc_id, superseded_by_doc_id. The sections array provides the content structure. Unit-level fields (heading_path, section_path, citation_anchor) are computed here from the content structure.

Stage 2: Chunking and Embedding — The Three Rules in Action

This is the most critical stage for metadata. The build_chunk_docs() function takes the CanonicalDocument and produces chunk dictionaries. Each chunk gets metadata treatment according to all three rules simultaneously.

Step 2a: Semantic Header (Rule 1 — Embed)

The semantic_header() function reads ONLY the Rule 1 (embedded) fields from the metadata spec and constructs a compact text prefix. This prefix gets prepended to the chunk text before the embedding model sees it.

Document A produces this header:

```
[OSFI | LAR | chapter_guideline | LAR (2026) Ch 2 > 2.1 Objective of the LCR]
```

Document B produces this header:

```
[OSFI | MCT | transmittal_letter]
```

> **Why this matters**
>
> The header shifts WHERE the chunk lands in vector space. A query for "OSFI LCR HQLA requirements" will land close to Doc A's chunks because the embedding sees "OSFI", "LAR", and the section heading. Doc B's chunks will be farther away because the embedding sees "transmittal_letter" and no LCR context. This is Rule 1 doing its job: metadata that changes semantic meaning.

Step 2b: Chunk Text Assembly

The semantic header is prepended to the paragraph text, then the combined string is sent to the embedding model (text-embedding-3-large, 1024 dimensions).

Document A, chunk for section 2.1:

```
chunk_text = "[OSFI | LAR | chapter_guideline | LAR (2026) Ch 2 > 2.1 Objective]"
             + "\n"
             + "This standard aims to ensure that an institution has an"
             + " adequate stock of unencumbered HQLA that consists of cash"
             + " or assets that can be converted into cash at little or no"
             + " loss of value in private markets..."

embedding = embed_texts([chunk_text])  -->  [0.023, -0.118, 0.042, ...]
                                            (1024-dim vector)
```

Long sections are split at 1,800 characters with 250-character overlap. Document A produces many chunks; Document B produces only a few.

Step 2c: Index Fields (Rule 2 — Store)

Each chunk dictionary also gets all Rule 2 fields attached as structured metadata. These fields are NOT in the chunk text — they are stored separately in Elasticsearch and PGVector as filterable columns.

Document A chunk row (key index fields):

```
{
```

```
  "citation_anchor": "osfi.lar.2026.chapter2.chapter::sec3::p2::chunk1",
  "status": "final_future_effective",
  "nova_tier": 1,
  "authority_class": "primary_normative",
  "authority_level": 100,
  "effective_date_start": "2026-05-01",
  "jurisdiction": "Canada",
  "sector": "Banks, Trust and Loan Companies",
  "current_version_flag": true,
  "contains_requirement": true,
  "contains_definition": false,
}
```

Document B chunk row (key index fields):

```
{
  "citation_anchor": "osfi.mct.2023.transmittal::sec2::p1::chunk1",
  "status": "superseded",
  "nova_tier": 2,
  "authority_class": "official_support",
  "authority_level": 60,
  "effective_date_start": "2023-10-20",
  "superseded_by_doc_id": "osfi.mct.2026.transmittal",
  "current_version_flag": false,
}
```

Why these fields are NOT embedded

"superseded" and "2023-10-20" do not change WHAT a paragraph is about. They change WHETHER it should be returned and HOW it should be ranked. Embedding them would pollute the vector space — making chunks about the same topic land in different places just because one is superseded. Rule 2 keeps these as structured filters instead.

Stage 3: Storage — Elasticsearch + PGVector

Each chunk is upserted into both stores with the exact same field structure. The primary key is citation_anchor.

Elasticsearch Mapping (relevant fields):

```
doc_id:                keyword       (filter)
title:                 text+keyword   (BM25 search + filter)
short_title:           keyword       (BM25 boost field)
status:                keyword       (filter: "final_current", "superseded"...)
effective_date_start: date           (range filter for as-of queries)
effective_date_end:   date           (range filter)
jurisdiction:          keyword       (filter: "Canada")
nova_tier:             integer       (boost/rank multiplier)
authority_class:       keyword       (filter)
sector:                keyword       (filter)
dense_vector:          dense_vector  (cosine similarity search)
chunk_text:            text          (BM25 full-text search)
```

PGVector Table (relevant columns):

```
CREATE TABLE nova_chunks (
  citation_anchor  TEXT PRIMARY KEY,
  doc_id           TEXT NOT NULL,
  chunk_text       TEXT NOT NULL,
  status           TEXT,
  effective_date_start  DATE,
  effective_date_end    DATE,
  jurisdiction     TEXT,
  nova_tier        INT,
  authority_class  TEXT,
  embedding        vector(1024)
);
```

Both stores hold the same data. Elasticsearch is used for hybrid BM25+dense retrieval with RRF fusion. PGVector provides a validation/augmentation path using pure vector similarity within PostgreSQL.

Stage 4: Retrieval — How Metadata Gates and Ranks

When a user asks: **"What is the current OSFI LCR requirement for Canadian banks?"** the retrieval system uses Rule 2 metadata at every step.

Step 4a: Query Filters (Rule 2 in action)

Before any similarity search runs, the retrieval notebook applies metadata filters:

```
filters = [
  {"term":  {"jurisdiction": "Canada"}},
  {"terms": {"status": ["active", "final_current", "future_effective"]}},
  {"terms": {"source_type": ["external_regulatory"]}},
  ... as_of_filter_clauses("2026-03-24")
]
```

What happens to our two documents at this stage

Document A (LAR Chapter 2): PASSES all filters. status = "final_future_effective" is in the allowed list. jurisdiction = "Canada" matches. It proceeds to similarity search.

Document B (MCT Letter): BLOCKED by the status filter. status = "superseded" is not in ["active", "final_current", "future_effective"]. It is excluded from results entirely. This is Rule 2 protecting answer quality — the model never even sees superseded content.

Step 4b: Hybrid Search (BM25 + Dense)

For documents that pass the filters, Elasticsearch runs two parallel retrievals fused with Reciprocal Rank Fusion (RRF):

- **BM25 path:** Searches chunk_text, title, and short_title for query terms. Doc A's chunks containing "LCR", "HQLA", "liquidity" score highly.
- **Dense vector path:** Compares the query embedding against the stored dense_vector. Doc A's chunks score highly because the Rule 1 semantic header placed them in the right region of vector space.
- **RRF fusion:** Combines ranks from both paths. Chunks appearing in both top-k lists get the strongest combined scores.

Step 4c: PGVector Augmentation

The retrieval notebook also queries PGVector independently with the same filters and fuses those results with the Elasticsearch results. This dual-store approach catches chunks that one engine might miss.

Stage 5: Prompt Assembly — Rule 3 in Action

The top-k fused results are now rendered into the LLM's prompt context. This is where Rule 3 metadata gets injected. The render_hit_for_prompt() function reads the prompt_injected_fields from the metadata spec and constructs a header for each chunk.

What the LLM sees for a Document A chunk:

```
TITLE: Liquidity Adequacy Requirements (LAR) (2026) Chapter 2 - LCR
CITATION_ANCHOR: osfi.lar.2026.chapter2.chapter::sec3::p2::chunk1
REGULATOR: OSFI
VERSION_ID: 2026
VERSION_LABEL: LAR (2026)
STATUS: final_future_effective
CURRENT_VERSION_FLAG: True
EFFECTIVE_DATE_START: 2026-05-01
AUTHORITY_CLASS: primary_normative
NOVA_TIER: 1
JURISDICTION: Canada
---
This standard aims to ensure that an institution has an adequate
stock of unencumbered HQLA that consists of cash or assets that
can be converted into cash at little or no loss of value...
```

Why Rule 3 matters for answer quality

Without this header, the LLM would see the paragraph text but would not know: (a) which regulator issued it, (b) whether it is current or superseded, (c) when it takes effect, (d) how authoritative it is, or (e) how to cite it. The model would produce answers that are undated, unattributed, and potentially mix current and outdated requirements. Rule 3 gives the LLM everything it needs to reason about authority, currency, and citation.

If Document B had passed retrieval (hypothetically):

For illustration, here is what the LLM would see. Note how the metadata immediately signals this is superseded and low-authority:

```
TITLE: 2024 Minimum Capital Test (MCT) - Letter (2023)
STATUS: superseded
CURRENT_VERSION_FLAG: False
EFFECTIVE_DATE_START: 2023-10-20
AUTHORITY_CLASS: official_support
NOVA_TIER: 2
---
Today, the Office of the Superintendent of Financial Institutions
(OSFI) is publishing the Minimum Capital Test (MCT) 2024 guideline...
```

The LLM can immediately see this is a superseded Tier 2 letter from 2023, and would correctly deprioritize it versus a current Tier 1 guideline.

5. Complete Metadata Field Reference

This table shows every metadata field, which Rule it serves, and what it does in the pipeline.

Field	Rule 1 Embed	Rule 2 Index	Rule 3 Prompt	Op Only	Description
doc_id	✓	✓			Stable document identifier
title		✓	✓		Full document title
short_title	✓	✓			Compact title for semantic header
document_class	✓	✓			chapter_guideline, transmittal_letter, etc.
heading_path	✓	✓			Section ancestry for each unit
section_path	✓	✓			Normalized section path string
citation_anchor		✓	✓		Unique chunk citation reference
regulator	✓	✓	✓		Regulatory authority (OSFI)
guideline_number	✓	✓			Official guideline code (LAR, CAR, MCT)
status		✓	✓		final_current, superseded, draft, etc.
current_version_flag		✓	✓		Whether this is the live version
effective_date_start		✓	✓		When the document takes effect
effective_date_end		✓	✓		When the document expires (if known)
authority_class		✓	✓		primary_normative, official_support, etc.
authority_level		✓			Numeric authority rank (100, 60, etc.)
nova_tier		✓	✓		NOVA tier for ranking (1, 2, 3)
jurisdiction		✓	✓		Canada
sector		✓			Banks, Insurance, Trust, etc.
version_id		✓	✓		Version identifier
version_label		✓	✓		Human-readable version label
supersedes_doc_id		✓			Prior version's doc_id
superseded_by_doc_id		✓			Next version's doc_id
raw_path				✓	Path to raw HTML in storage
canonical_json_path				✓	Path to canonical JSON in storage
raw_sha256				✓	Hash for change detection
parser_version				✓	Which parser version produced this
quality_score				✓	Extraction quality (0.2 to 1.0)

6. Visual Summary: Metadata Flow Through the Pipeline

Here is the complete flow of one Document A chunk through all five stages, showing exactly which metadata is used where:

```
STAGE 1: PARSING
──────────────────────────────────────────────────────────────────────────
JSON file --> parse_osfi_canonical_json() --> CanonicalDocument
  Reads: doc_id, title, sections[], status, nova_tier, all 121 fields
  Computes: heading_path, section_path, citation_anchor (unit-level)

                              |
                              v

STAGE 2: CHUNKING + EMBEDDING
──────────────────────────────────────────────────────────────────────────
build_chunk_docs(doc) splits content into chunks, applies 3 rules:

  RULE 1 (Embed):    semantic_header() reads embedded_fields from spec
                     --> "[OSFI | LAR | chapter_guideline | Ch2 > 2.1]"
                     --> prepended to chunk text --> sent to embedding model

  RULE 2 (Index):    Attaches status, nova_tier, effective_date_start...
                     as structured fields on the chunk dict

  Operational:       Attaches raw_path, sha256, parser_version

                              |
                              v

STAGE 3: STORAGE
──────────────────────────────────────────────────────────────────────────
Upsert chunk dict to BOTH stores:
  Elasticsearch: keyword fields for filtering + dense_vector for ANN
  PGVector:      SQL columns for filtering + embedding for <=> distance

                              |
                              v

STAGE 4: RETRIEVAL (at query time)
──────────────────────────────────────────────────────────────────────────
  RULE 2 (Filter):  WHERE status IN ("final_current","future_effective")
                    AND jurisdiction = "Canada"
                    AND effective_date_start <= "2026-03-24"
  RULE 2 (Rank):    RRF fusion of BM25 (uses chunk_text) + dense (uses vector)

                              |
                              v
```

```
STAGE 5: PROMPT ASSEMBLY (at answer time)
──────────────────────────────────────────────────────────────────────────────
  RULE 3 (Inject):  render_hit_for_prompt() reads prompt_injected_fields
                    --> TITLE: ..., STATUS: ..., NOVA_TIER: ...
                    --> Rendered as human-readable header above chunk text
                    --> LLM uses this to reason about authority and currency
```

The Anti-Pattern to Avoid

Never embed Rule 2 fields (status, effective_date, nova_tier) into chunk text. The bad header looks like: [status=superseded | tier=1 | effective=2025-04-01 | parser_version=v2.1.0]. This pollutes the embedding space and makes chunks about the same topic land in different vector positions just because their administrative metadata differs.